

Objetivo

O objetivo desta atividade prática em laboratório é implementar e testar algoritmos de **busca em vetor**, com foco na **busca binária**. A atividade deve reforçar a ideia central vista em sala: a busca binária só pode ser aplicada corretamente quando os dados estão em um **vetor ordenado**.

Ao final, o programa deverá comparar o comportamento da busca linear, da busca linear em vetor ordenado e da busca binária, observando a quantidade de comparações realizadas em cada caso.

Parte 1 – Implementação dos Algoritmos de Busca

1. Inicialmente crie os seguintes arquivos fonte:

- **ResultadoBusca**: classe que representa o resultado de uma busca;
- **Busca**: classe que implementa os algoritmos de busca e funções auxiliares;
- **BuscaMain**: classe com o método principal para demonstrar o funcionamento dos algoritmos.

2. Estrutura sugerida para a classe **ResultadoBusca**:

```
public class ResultadoBusca {  
    private int indice;  
    private int comparacoes;  
  
    public ResultadoBusca(int indice, int comparacoes) {  
        this.indice = indice;  
        this.comparacoes = comparacoes;  
    }  
  
    // getters  
}
```

3. Significado dos atributos:

- **indice**: posição em que o elemento foi encontrado, ou -1 caso o elemento não esteja no vetor;
- **comparacoes**: quantidade de comparações realizadas entre o elemento procurado e elementos do vetor.

4. Métodos a serem implementados na classe **Busca**:

- (a) `public int[] criaVetorOrdenado(int n)`: cria um vetor ordenado com **n** elementos;

- (b) `public void imprimeVetor(int[] vet):` imprime os elementos do vetor;
 - (c) `public ResultadoBusca buscaLinear(int[] vet, int elem):` percorre o vetor elemento a elemento até encontrar `elem` ou até chegar ao final;
 - (d) `public ResultadoBusca buscaLinearOrdenada(int[] vet, int elem):` percorre o vetor ordenado e interrompe a busca quando encontrar um valor maior que `elem`;
 - (e) `public ResultadoBusca buscaBinaria(int[] vet, int elem):` implementa a busca binária de forma iterativa;
 - (f) `public ResultadoBusca buscaBinariaRecursiva(int[] vet, int elem):` método público que chama a versão recursiva da busca binária;
 - (g) `private ResultadoBusca buscaBinariaRecursiva(int[] vet, int ini, int fim, int elem, int[] aux):` método auxiliar recursivo que realiza a busca no intervalo `ini..fim`.
5. O método `criaVetorOrdenado` pode preencher o vetor com valores crescentes simples, por exemplo:

```
vet[i] = i * 2;
```

Essa estratégia facilita a criação de testes com elementos existentes e inexistentes no vetor.

6. A busca binária iterativa deve seguir a lógica vista em sala:

```
ini = 0;
fim = vet.length - 1;

enquanto ini <= fim:
    meio = ini + (fim - ini) / 2;

    se elem < vet[meio]:
        fim = meio - 1;
    senao se elem > vet[meio]:
        ini = meio + 1;
    senao:
        retorna meio;

retorna -1;
```

7. A busca binária recursiva deve usar o seguinte caso de parada:

```
se ini > fim:
    retorna -1;
```

Caso contrário, calcule o meio do intervalo, compare o elemento procurado com `vet[meio]` e faça a chamada recursiva apenas na metade que ainda pode conter o elemento.

8. Não aplique busca binária em vetor desordenado. A ordenação do vetor é uma pré-condição para a correção do algoritmo.

Parte 2 – Testes com Vetores Pequenos

Na classe `BuscaMain`, crie vetores pequenos para conferir se os algoritmos retornam os índices corretos.

1. Crie um vetor ordenado com 10 elementos e imprima seu conteúdo;

2. Teste a busca por pelo menos três elementos:
 - um elemento que esteja no início do vetor;
 - um elemento que esteja no meio do vetor;
 - um elemento que não esteja no vetor.
3. Para cada teste, execute:
 - busca linear;
 - busca linear em vetor ordenado;
 - busca binária iterativa;
 - busca binária recursiva.
4. Para cada execução, imprima:
 - o valor procurado;
 - o índice retornado;
 - a quantidade de comparações realizadas.

Parte 3 – Experimento com Tamanhos Maiores

Implemente um experimento para observar a diferença de comportamento entre busca linear e busca binária.

1. Crie vetores ordenados com os seguintes tamanhos:

10, 100, 1000, 10000, 100000
2. Para cada tamanho, procure por um elemento que esteja no final do vetor;
3. Para cada tamanho, procure também por um elemento que não esteja no vetor;
4. Registre e imprima a quantidade de comparações realizadas por cada algoritmo;
5. Organize a saída em formato de tabela, por exemplo:

n	elem	linear	linear_ord	binaria	binaria_rec
10	18	...	...	...	...
100	198	...	...	...	...
1000	1998	...	...	...	...

Questões para Análise

Após executar os testes, responda no próprio código, em forma de comentários, ou em um pequeno arquivo de texto:

1. Por que a busca binária exige que o vetor esteja ordenado?
2. Em quais casos a busca linear em vetor ordenado pode parar antes de chegar ao final do vetor?
3. O que acontece com a quantidade de comparações da busca linear quando **n** aumenta?

4. O que acontece com a quantidade de comparações da busca binária quando **n** aumenta?
5. Para os testes realizados, a busca binária iterativa e a busca binária recursiva retornaram os mesmos índices? Explique.

Observações

- Use vetores de inteiros;
- Use `ini + (fim - ini) / 2` para calcular o meio do intervalo;
- O método deve retornar `-1` quando o elemento não for encontrado;
- A contagem de comparações deve considerar comparações entre **elem** e algum elemento do vetor;
- Faça testes com elementos existentes e inexistentes no vetor;
- Utilize adequadamente os princípios de encapsulamento da programação orientada a objetos.

Observação Final

Ao final da atividade, o programa deve demonstrar claramente que a busca binária reduz o espaço de busca pela metade a cada iteração ou chamada recursiva, apresentando desempenho muito superior ao da busca linear em vetores grandes e ordenados.